

# Cross-Platform Network Hardening Guide

Disabling IPv6, Overriding Gateway DNS, and Enforcing Quad9 across Linux, Windows, and macOS

## 1. Architectural Constraints for Privacy

Disabling the IPv6 stack on an operating system forces all network transactions onto the IPv4 protocol. However, data aggregators, tracker scripts, and telemetry collectors (such as Google and Facebook) maintain extensive dual-stack network infrastructure. If IPv6 is unavailable, communication links fail back to IPv4 transparently. To successfully counter cross-site tracking and operating system telemetry, connections must be blocked at the **DNS resolution layer**.

Additionally, modern residential ISP gateways broadcast their own global or link-local IPv6 addresses through **Router Advertisements (RA)**. This forces client machines to append the ISP's gateway to their active DNS list, causing tracking queries to slip past custom settings. Below are the definitive methods to disable IPv6 and lock in secure DNS across all major platforms.

## 2. Linux Configurations

### Kernel-Level IPv6 Suppression (Permanent)

1. Open or create a system runtime configuration drop-in file:

```
sudo nano /etc/sysctl.d/99-disable-ipv6.conf
```

2. Inject the following lines to completely bind the network kernel strictly to IPv4:

```
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1
```

3. Load and commit the changes live without restarting parameters:

```
sudo sysctl --system
```

### Enforcing an Immutable DNS Resolver Lock

1. Edit the core system resolver layout:

```
sudo nano /etc/resolv.conf
```

2. Strip out all dynamic parameters and input the clean, unfiltered Quad9 addresses strictly:

```
nameserver 9.9.9.9
nameserver 149.112.112.112
nameserver 2620:fe::fe
nameserver 2620:fe::9
```

3. Enforce the immutable system flag. This locks the file completely, throwing immediate access errors if the router, NetworkManager, or background dynamic DHCP daemons attempt to overwrite your settings:

```
sudo chattr +i /etc/resolv.conf
```

4. Flush the internal configuration engines:

```
sudo systemctl restart NetworkManager
```

---

## 3. Windows 11 Configurations

### Disabling IPv6 via Network Connections

To drop the IPv6 protocol stack across physical network boundaries using standard administrative structures:

1. Trigger the Run prompt by pressing **Win + R**, input `ncpa.cpl`, and press **Enter** to land on the classic Network Connections menu.
2. Identify your active adapter (e.g., *Ethernet* or *Wi-Fi*), right-click, and open **Properties**.
3. Navigate through the binding list to locate **Internet Protocol Version 6 (TCP/IPv6)**.
4. **Uncheck** the item, click **OK**, and reboot the system to flush dynamic interface allocations.

### Overriding Gateway Advertisements and Forcing Quad9 DNS

To bypass automated ISP gateway DNS injection and assign standard servers directly through the command shell:

1. Open an elevated PowerShell session (Right-click Start -> **Terminal (Admin)** or **PowerShell (Admin)**).

2. Locate the precise name of your active physical connection wrapper:

```
Get-NetAdapter
```

3. Force static DNS assignments onto the interface (substitute "Ethernet" with your exact interface name from step 2). This overrides and blocks any dynamic DNS configurations pushed by the router:

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses ("9.9.9.9", "149
```

4. Clear the local system client resolution resolver caches entirely to apply the adjustments:

```
Clear-DnsClientCache
```

---

## 4. macOS Configurations

### Disabling IPv6 via Command Line (Terminal)

Modern macOS architectures require disabling IPv6 assignments explicitly across your active hardware ports via the network setup utility.

1. Launch the native Terminal tool and list the exact hardware interfaces recognized by the system:

```
networksetup -listallnetworkservices
```

2. Deactivate IPv6 handling on your active network service ports. For systems relying entirely on wireless configurations, explicitly target the "Wi-Fi" interface profile:

```
# For Wi-Fi Dedicated Environments
sudo networksetup -setv6off "Wi-Fi"

# For Wired or Docked Environments
sudo networksetup -setv6off "Ethernet"
```

*Note: If the shell yields an error locating the resource name, inspect your system-specific labels via `networksetup -listallnetworkservices`.*

### Locking Down Hardcoded DNS Infrastructure

To isolate name resolution from residential gateway injection protocols and lock down Quad9:

1. Explicitly point your interface network service parameters directly toward the secure upstream targets:

```
# For Wi-Fi Dedicated Environments
sudo networksetup -setdnsservers "Wi-Fi" 9.9.9.9 149.112.112.112

# For Wired or Docked Environments
sudo networksetup -setdnsservers "Ethernet" 9.9.9.9 149.112.112.112
```

2. Purge the active directory name system caches internally to force immediate system alignment:

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

---

## 5. Tailscale Mesh Network Compatibility

Implementing these configuration steps maintains full **Tailscale Mesh Network (Tailnet)** accessibility and data integrity across all platforms due to the following structural engineering factors:

- **Independent Stack Architecture:** Tailscale builds an isolated encrypted overlay adapter on the host machine. While physical interfaces are restricted strictly to IPv4 pathways, Tailscale handles internal machine-to-machine traffic securely via virtualized IPv4 ( `100.x.y.z` ) and internal loopback IPv6 scopes ( `fd7a:...` ) by encapsulating packets into outbound physical IPv4 links.
- **Targeted DNS Separation (MagicDNS):** The system hardening directives modify physical port environments exclusively. Tailscale preserves its internal resolution engine boundaries by routing local mesh keywords (tagged with the `~.` routing flag or terminating in `.ts.net` ) directly to the internal loopback resolver at `100.100.100.100` , while generic external connections push cleanly to the hardened Quad9 tier.

---

## 6. Verification Checklists

Once deployment across your systems is complete, execute these specific diagnostics to ensure all rogue router injections are closed:

- **Linux:** Verify that `resolvectl status` lists `resolv.conf mode: foreign` and displays zero references to your ISP's residential prefix (e.g., `2607:...` ).

- **Windows:** Launch an administrative PowerShell prompt and execute `Get-DnsClientServerAddress`. Check that your physical interfaces explicitly restrict their server listings to Quad9.
- **macOS:** Query your active resolvers by running `scutil --dns` inside the terminal. The top configuration blocks should reflect your manually assigned DNS servers exclusively.